

**Szegedi Tudományegyetem
Informatikai Intézet**

SZAKDOLGOZAT

Kökény Szabolcs

2026

**Szegedi Tudományegyetem
Informatikai Intézet**

Jegyzetmegosztó Rendszer Study Hub

Szakdolgozat

Készítette:
Kökény Szabolcs
programtervező informatikus
BSc hallgató

Témavezető:
Dr. Bilicki Vilmos
egyetemi adjunktus

Szeged
2026

Feladatkiírás

Egy olyan webalapú platform megvalósítása teljes dokumentációval, amely lehetővé teszi egyetemi hallgatók számára tananyagok, például jegyzetek, gyakorló feladatok és egyéb oktatási segédanyagok megosztását és elérését.

A rendszernek képesnek kell lennie a feltöltött tartalmak strukturált kezelésére egyetem, kar és kurzus szerint, valamint biztosítania kell a hatékony keresést és szűrést a releváns tananyagok gyors megtalálása érdekében. A felhasználók számára lehetőséget kell nyújtani a tartalmak értékelésére és véleményezésére, ezzel támogatva a közösségi alapú visszajelzést és a minőségi anyagok kiemelését.

A rendszernek kezelnie kell a felhasználói profilokat és a feltöltött tartalmakat, valamint biztosítania kell a jogosultságkezelést és a biztonságos hozzáférést.

A megvalósítás során egy korszerű webes architektúra kerül alkalmazásra, amely frontend és backend komponensekből áll, valamint relációs adatbázist használ az adatok tárolására. A rendszernek alkalmasnak kell lennie arra, hogy elősegítse az egyetemi hallgatók közötti tudásmegosztást és hatékonyabb tanulást.

Tartalmi összefoglaló

Téma megnevezése:

Jegyzetmegosztó Rendszer - Study Hub

A megadott feladat megfogalmazása:

A feladat egy olyan webalkalmazás létrehozása, amely lehetővé teszi egyetemi hallgatók számára tananyagok, például jegyzetek, gyakorló feladatok és egyéb oktatási segédanyagok feltöltését, rendszerezését és megosztását más felhasználókkal, valamint a közös tanulás elősegítését.

Megoldási mód:

A feladat megoldása során egy modern, kliens-szerver architektúrára épülő webalkalmazás került kialakításra. A frontend megvalósítása React [1] keretrendszer segítségével történt, míg a backend oldali logika Node.js [2] és Express [3] környezetben került implementálásra. Az adatok tárolására MySQL [4] relációs adatbázis szolgál.

Alkalmazott eszközök, módszerek:

A fejlesztés Visual Studio Code [5] fejlesztői környezetben történt. A frontend oldalon React [1] keretrendszert, míg a backend oldalon Node.js [2] és Express [3] technológiákat alkalmaztam. Az adatkezelés MySQL [4] adatbázissal valósult meg, a frontend és backend közötti kommunikáció REST API-n keresztül történik.

Elért eredmények:

Az elkészült webalkalmazás lehetőséget biztosít a felhasználók számára tananyagok feltöltésére, böngészésére és keresésére egyetem, kar és kurzus szerinti bontásban. A rendszer támogatja a felhasználók közötti interakciót hozzászólások és értékelések formájában, valamint biztosítja a felhasználói profilok kezelését. Az alkalmazás reszponzív kialakításának köszönhetően különböző eszközökön, asztali számítógépen, laptopon és mobil eszközökön is hatékonyan használható.

Kulcsszavak:

webalkalmazás, tananyagmegosztás, React [1], Node.js [2], MySQL [4], REST API

Tartalomjegyzék

FELADATKIÍRÁS.....	1
TARTALMI ÖSSZEFOGLALÓ.....	2
Téma megnevezése:	2
A megadott feladat megfogalmazása:	2
Megoldási mód:.....	2
Alkalmazott eszközök, módszerek:.....	2
Elért eredmények:	2
Kulcsszavak:.....	2
TARTALOMJEGYZÉK	3
1. BEVEZETÉS.....	5
2. PIACKUTATÁS.....	6
2.1. Notion [6]	6
2.2. Google Classroom [7]	7
2.3. Studocu [8].....	7
2.4. Studydrive [9]	7
3. TERVEZETT MEGJELENÉS ÉS FELHASZNÁLÓI FELÜLET	8
3.1. Tervezési alapelvek.....	8
3.2. Mobile-first megközelítés	8
3.3. Főbb felületek bemutatása	9
3.3.1. Bejelentkezés és regisztráció	9
3.3.2. Főoldal	10
3.3.3. Kurzusok.....	11
3.3.4. Kurzus jegyzetek.....	12
3.3.5. Jegyzet részletek.....	12
3.3.6. Beállítások	13
3.3.7. Jegyzet létrehozása.....	14
4. HASZNÁLT TECHNOLÓGIÁK.....	14
4.1. React [1]	14
4.1.1. Vite [13], Typescript [14]	15
4.1.2. Főbb frontend kiegészítő könyvtárak.....	15
4.1.3. Node.js [2]	16

4.2.	Express [3], TypeScript [14].....	16
4.2.1.	Főbb backend kiegészítő könyvtárak.....	16
4.3.	Miért ezeket a technológiákat választottam?	17
5.	RENDSZERACHITEKTÚRA	18
5.1.	Frontend architektúra	18
5.2.	Backend architektúra.....	19
5.3.	Adatbázis architektúra.....	20
6.	KÓDRÉSZLETEK.....	21
6.1.	Felhasználó bejelentkezés.....	21
6.2.	Jelszó módosítása	22
6.3.	Jegyzet létrehozása	23
6.4.	Hozzászólás írása jegyzethez.....	23
6.5.	Kliensoldali navigáció (Routing)	24
6.6.	API kommunikáció.....	25
7.	TELEPÍTÉSI ÚTMUTATÓ.....	25
7.1.	Környezet.....	25
7.2.	Backend beállítása	26
7.3.	Frontend beállítása	27
8.	TOVÁBBFEJLESZTÉSI LEHETŐSÉGEK	27
9.	MESTERSÉGES INTELLIGENCIA HASZNÁLATA A FEJLESZTÉS SORÁN	28
9.1.	Alkalmazott eszközök	28
9.2.	Felhasználási területek	29
9.3.	Validáció és ellenőrzés	29
9.4.	MI használatának korlátai	30
9.5.	Tapasztalatok és tanulságok.....	30
	IRODALOMJEGYZÉK	31
	NYILATKOZAT	32
	KÖSZÖNETNYILVÁNÍTÁS.....	33

1. Bevezetés

Az információs technológia fejlődésével párhuzamosan az oktatás és a tanulási szokások is jelentős átalakuláson mentek keresztül. Az egyetemi hallgatók mindennapjainak szerves részévé vált a digitális tananyagok használata, legyen szó előadásjegyzetéről, gyakorló feladatokról vagy vizsgafelkészítő anyagokról. Azonban ezek az anyagok gyakran különböző platformokon, rendszereken és kommunikációs csatornákon szétszórva érhetők el, ami megnehezíti a hatékony tanulást és az információk gyors elérését.

A jelenlegi gyakorlatban a hallgatók sok esetben informális csatornákon keresztül osztják meg egymással a tananyagokat, például közösségi média csoportokban, üzenetküldő alkalmazásokban vagy felhőalapú tárhelyeken. Ezek a megoldások bár rövid távon működőképesek, hosszabb távon több problémát is felvetnek. Ilyen problémák például a tartalmak rendszerezésének hiánya, a nehézkes kereshetőség, az információk elvesztésének kockázata, valamint az, hogy nincs lehetőség a tananyagok minőségének értékelésére vagy visszajelzés adására.

A szakdolgozat célja egy olyan webalapú rendszer, a Study Hub megtervezése és megvalósítása, amely központosított felületet biztosít az egyetemi tananyagok megosztására és kezelésére. A rendszer lehetőséget ad a felhasználóknak arra, hogy tanulást segítő anyagokat töltsenek fel, azokat strukturált formában egyetem, kar és kurzus szerint rendszerezék, valamint egyszerűen és gyorsan hozzáférjenek mások által megosztott tartalmakhoz.

A Study Hub egyik kiemelt célja a közösségi tanulás támogatása. Ennek érdekében a rendszer nem csupán passzív tartalomfogyasztást tesz lehetővé, hanem interaktív funkciókat is biztosít, mint például a hozzászólás és az értékelés. Ezek a funkciók elősegítik a felhasználók közötti kommunikációt, valamint hozzájárulnak ahhoz, hogy a magasabb minőségű tananyagok könnyebben kiemelkedjenek.

A dolgozat során egy korszerű webes technológiákra épülő alkalmazás kerül bemutatásra. A rendszer kliensoldali része React [1] keretrendszer segítségével valósult meg, amely lehetővé teszi dinamikus és felhasználóbarát felületek kialakítását. A szerveroldali logika Node.js és Express környezetben került implementálásra, míg az adatok tárolása MySQL [4] relációs adatbázisban történik. A frontend és backend közötti kommunikáció REST API-n keresztül valósul meg.

A dolgozat további részeiben bemutatásra kerülnek a rendszerrel szemben támasztott követelmények, a tervezési folyamat, az adatbázis felépítése, valamint a megvalósítás részletei. Ezt követően ismertetésre kerülnek az elkészült rendszer működési jellemzői és az elért eredmények, végül pedig összegzésre kerülnek a fejlesztés tapasztalatai és a lehetséges jövőbeli bővítési irányok.

A Study Hub célja, hogy hozzájáruljon az egyetemi hallgatók tanulási folyamatának hatékonyabbá tételéhez, valamint egy olyan közösségi platformot biztosítson, amely elősegíti a tudásmegosztást és az együttműködést a hallgatók között.

2. Piackutatás

A Study Hub rendszer tervezése során fontos szempont volt a már létező, oktatási célú és tudásmegosztást támogató platformok elemzése. A piackutatás célja annak feltérképezése volt, hogy milyen megoldások érhetők el jelenleg, ezek milyen funkciókat kínálnak, illetve milyen hiányosságok figyelhetők meg bennük. Az elemzés eredményei hozzájárultak egy olyan rendszer kialakításához, amely a meglévő megoldások előnyeit ötvözi, miközben azok korlátait igyekszik csökkenteni.

A vizsgálat során több, széles körben használt platform került elemzésre, különös tekintettel a felhasználói felület kialakítására, a tartalomkezelési megoldásokra, valamint a közösségi funkciókra.

2.1. Notion [6]

A Notion [6] egy általános célú jegyzetelési és tudásmenedzsment rendszer, amely kiemelkedik moduláris felépítésével és letisztult felhasználói felületével. A platform lehetőséget biztosít különböző típusú tartalmak strukturált kezelésére, valamint azok hierarchikus rendezésére.

A StudyHub tervezése során a Notion [6] minimalista megjelenése és kártya alapú elrendezése jelentős inspirációt nyújtott. Különösen fontos szempont volt a felhasználói figyelem fókuszálása, azaz egy olyan felület kialakítása, amely mentes a zavaró elemek nagy részétől, és a tananyagok gyors elérését támogatja.

Ugyanakkor a Notion [6] rendkívül rugalmas és sokféleképpen testre szabható rendszere a Study Hub esetében egyszerűsítésre került. A cél egy olyan célzott alkalmazás létrehozása volt, amely kifejezetten tananyagmegosztásra optimalizált, így a komplex konfigurációs lehetőségek helyett az egyszerű használhatóság került előtérbe.

2.2. Google Classroom [7]

A Google Classroom [7] elsősorban oktatók és hallgatók közötti kommunikációt és tananyagkezelést támogató rendszer. A platform egyik legfontosabb jellemzője a kurzus alapú szervezési modell, amelyben a tananyagok egyértelműen strukturált módon jelennek meg.

A Study Hub rendszerben ez a kurzus alapú megközelítés szintén megjelenik, hiszen a feltöltött anyagok egyetem, kar és kurzus szerint kerülnek rendszerezésre. Ez a struktúra lehetővé teszi a felhasználók számára, hogy gyorsan navigáljanak a releváns tartalmak között.

Fontos különbség azonban, hogy míg a Google Classroom [7] egy tanárközpontú rendszer, addig a Study Hub alapvetően hallgatók közötti tudásmegosztásra épül. Ennek megfelelően a tartalmak létrehozása és kezelése nem kizárólag oktatókhoz kötött, hanem minden felhasználó számára elérhető.

2.3. Studocu [8]

A Studocu [8] egy kifejezetten tananyagmegosztásra specializált platform, amely lehetővé teszi dokumentumok feltöltését, böngészését és keresését különböző kurzusokhoz kapcsolódóan. A rendszer egyik fő erőssége a dokumentum alapú tartalomkezelés és a keresési lehetőségek.

A Study Hub fejlesztése során a Studocu [8] megoldásai inspirálták a tananyagok listázásának és megjelenítésének módját, különösen a dokumentumok előnézeti (preview) megjelenítését és a gyors hozzáférést biztosító felületeket.

Ugyanakkor a Study Hub esetében a cél egy kevésbé kereskedelmi jellegű platform létrehozása volt. Ennek megfelelően a rendszer egyszerűbb, átláthatóbb felhasználói felületet alkalmaz, és nem tartalmaz olyan elemeket, mint például reklámok vagy fizetős hozzáférési modellek.

2.4. Studydrive [9]

A Studydrive [9] egy közösségi alapú tanulási platform, amely a hallgatók közötti együttműködésre és tudásmegosztásra helyezi a hangsúlyt. A rendszer lehetőséget biztosít különböző kurzusokhoz kapcsolódó anyagok megosztására és közösségi interakciókra.

A Study Hub [9] esetében ez a közösségi megközelítés szintén meghatározó szerepet játszik. A felhasználók nem csupán tartalomfogyasztók, hanem aktív résztvevői

a rendszernek, akik hozzászólásokkal és értékelésekkel járulhatnak hozzá a tartalmak minőségének javításához.

Eltérésként megemlíthető, hogy a Study Hub a közösségi funkciókat visszafogottabb formában alkalmazza, és a hangsúlyt elsősorban a tananyagok egyszerű és hatékony elérésére helyezi.

3. Tervezett megjelenés és felhasználói felület

A Study Hub rendszer felhasználói felületének kialakítása során elsődleges szempont volt az egyszerű használhatóság, az átláthatóság, valamint a gyors navigáció biztosítása. A tervezési folyamat nem egy előre elkészített, részletes prototípus (például Figma terv) [10] alapján történt, hanem iteratív módon, a fejlesztéssel párhuzamosan alakult ki. Ez a megközelítés lehetővé tette, hogy a felhasználói felület folyamatosan igazodjon a fejlesztés során felmerülő funkcionális igényekhez és gyakorlati tapasztalatokhoz.

3.1. Tervezési alapelvek

A felhasználói felület kialakítása során a következő alapelvek kerültek meghatározásra:

Egyszerűség: A rendszer célcsoportja egyetemi hallgatók, ezért fontos volt, hogy a felület gyorsan tanulható és könnyen használható legyen.

Átláthatóság: A tartalmak strukturált megjelenítése kiemelt szerepet kapott, különösen a kurzusokhoz és tananyagokhoz kapcsolódó nézetekben.

Fókuszált megjelenítés: A felület minimalista kialakítású, amely csökkenti a zavaró elemek számát, és elősegíti a tanulásra való koncentrációt.

Konzisztencia: Az egyes oldalak és komponensek egységes vizuális és működési logikát követnek.

3.2. Mobile-first megközelítés

A rendszer felhasználói felületének tervezése során a mobile-first elv került alkalmazásra. Ennek megfelelően a felület elsődlegesen mobil eszközökre lett optimalizálva, majd ezt követően került kiterjesztésre nagyobb képernyőméretekre, például tabletekre és asztali számítógépekre.

Ez a megközelítés különösen indokolt, mivel a hallgatók jelentős része mobil eszközön keresztül éri el a tananyagokat. A mobile-first tervezés előnye, hogy a

legfontosabb funkciók és tartalmak már kis képernyőn is jól használható formában jelennek meg, ami hozzájárul a jobb felhasználói élményhez.

A reszponzív kialakítás biztosítja, hogy a felület különböző eszközökön és képernyőméreteken is megfelelően jelenjen meg, a layout pedig dinamikusan alkalmazkodik az adott környezethez.

3.3. Főbb felületek bemutatása

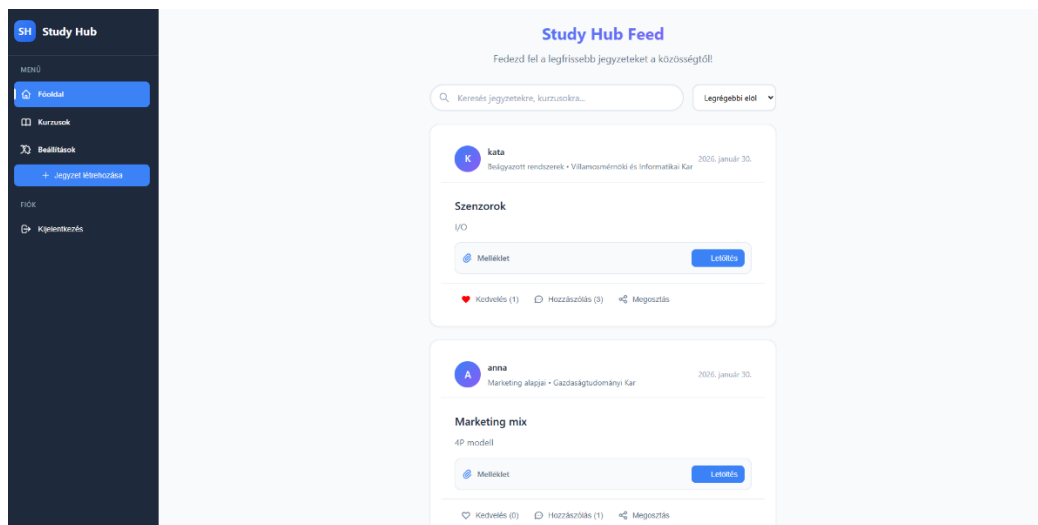
3.3.1. Bejelentkezés és regisztráció

ábra 3.3-1 Bejelentkezés

ábra 3.3.1-2 Regisztráció

A bejelentkezési és regisztrációs felület biztosítja a felhasználók számára a rendszerhez való hozzáférést. A felhasználók hagyományos módon, e-mail cím és jelszó megadásával hozhatnak létre fiókot, illetve jelentkezhetnek be. Emellett lehetőség van külső hitelesítési szolgáltatás használatára is, például Google-fiókkal [11] történő bejelentkezésre, amely egyszerűsíti és gyorsítja a folyamatot.

3.3.2. Főoldal

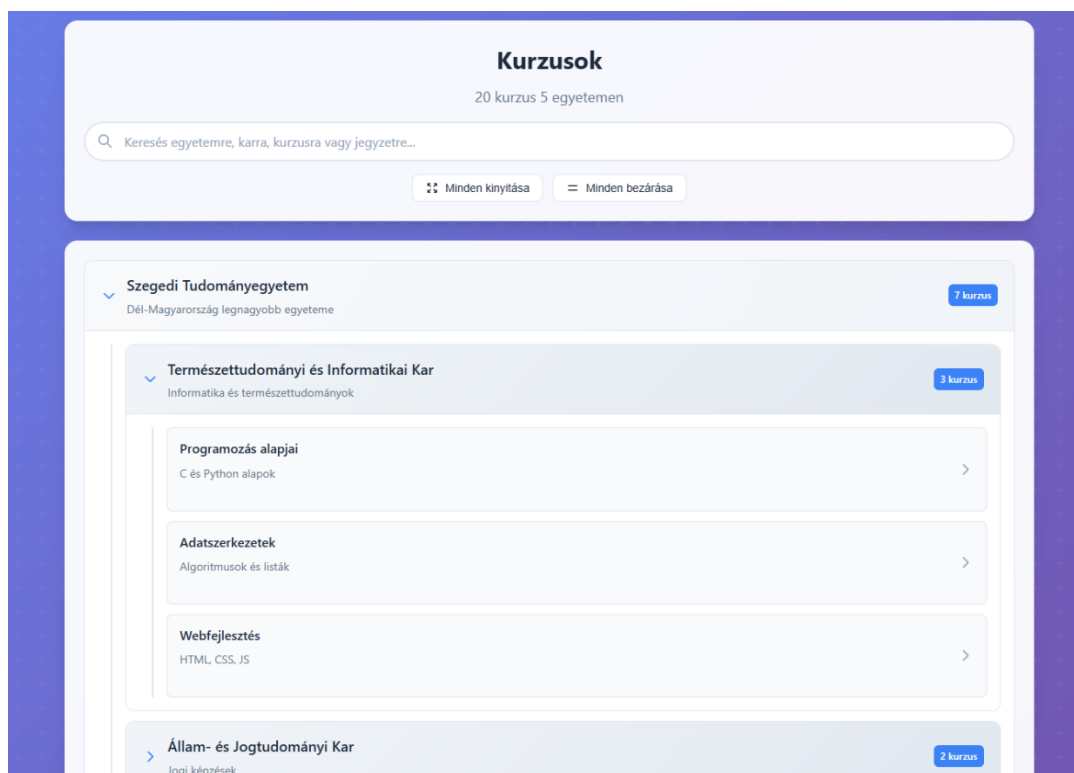


ábra 3.3-2Főoldal

A főoldal a rendszer központi felülete, ahol a felhasználók a közzétett tananyagokat tekinthetik meg. Az oldal bal oldalán egy navigációs sáv található, amely gyors hozzáférést biztosít a rendszer különböző részeihez.

A fő tartalmi rész egy úgynevezett „feed” nézetet tartalmaz, ahol a felhasználók által feltöltött jegyzetek posztok formájában jelennek meg. A felhasználók lehetőséget kapnak a tartalmak szűrésére és keresésére, ezáltal könnyebben megtalálhatják a számukra releváns tananyagokat.

3.3.3. Kurzusok

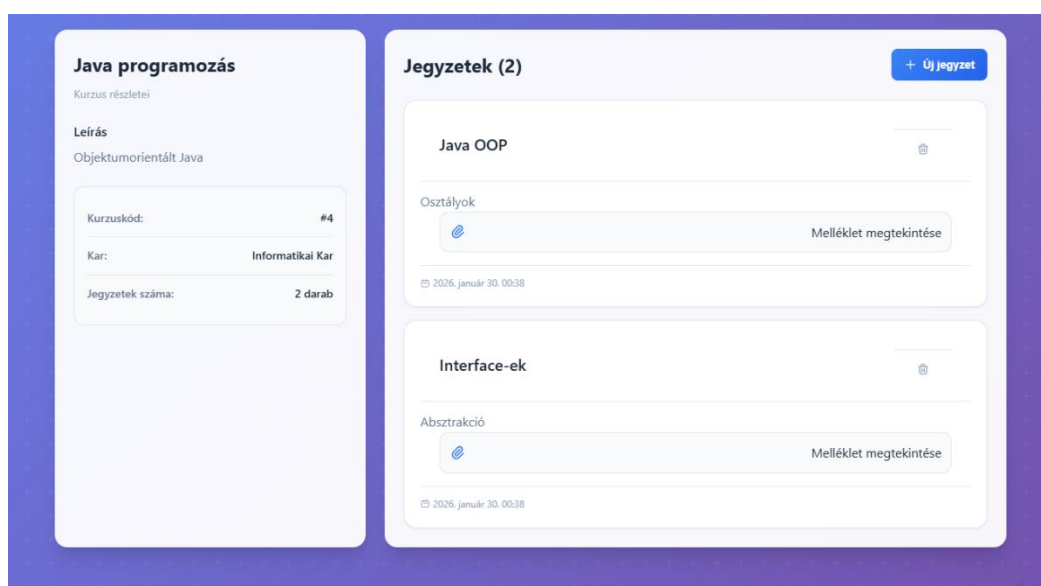


ábra 3.3-3Kurzusok

A kurzusok oldal a tananyagok strukturált böngészését teszi lehetővé. A tartalmak hierarchikus, lenyitható fa struktúrában jelennek meg, amely az egyetem, kar, szak és kurzus szintek szerinti felépítést követi.

Ez a megjelenítési forma segíti a felhasználókat abban, hogy gyorsan eligazodjanak a rendszerben, és könnyen megtalálják a számukra releváns kurzusokat, valamint az azokhoz kapcsolódó tananyagokat.

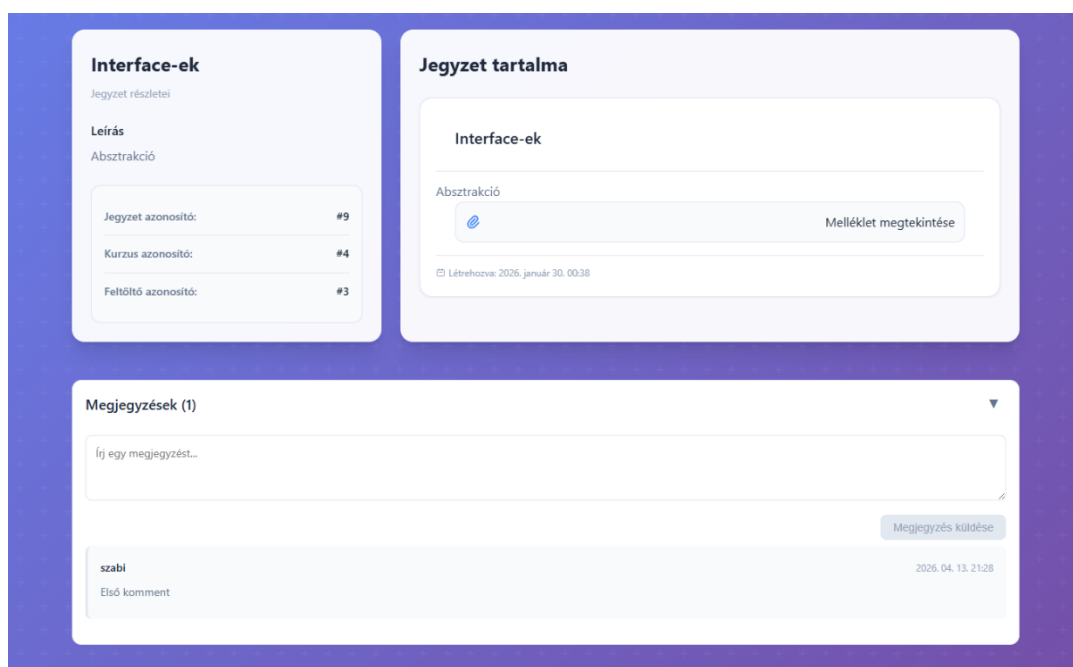
3.3.4. Kurszus jegyzetek



ábra 3.3-4Kurszus jegyzetek

Ezen a felületen egy adott kurzushoz tartozó jegyzetek listája jelenik meg. A felhasználók itt áttekinthetik az adott kurzushoz feltöltött tananyagokat, valamint láthatják azok számát is. A lista lehetővé teszi a gyors böngészést, és közvetlen hozzáférést biztosít az egyes jegyzetek részletes nézetéhez.

3.3.5. Jegyzet részletek



ábra 3.3-5Jegyzet részletek

A jegyzet részletek oldalon a kiválasztott tananyag teljes tartalma megjelenik. Itt látható a jegyzet címe, leírása, valamint a hozzá csatolt fájlok (például PDF,

dokumentumok vagy prezentációk). A felhasználók lehetőséget kapnak arra, hogy hozzászólásokat írjanak a jegyzethez, ezáltal támogatva a közösségi interakciót és a tapasztalatok megosztását.

3.3.6. Beállítások

Beállítások

Kezeld a fiókod beállításait és jelszavadat

Fiók információk
Áttekintés a fiókod adatairól

Email cím: szabi@gmail.com

Fiók létrehozva: 2026. 01. 30. 0:37

Utolsó bejelentkezés: 2026. 04. 28. 18:18

Jelszó módosítása
Változtasd meg a jelszavadat a fiókod biztonsága érdekében

Jelenlegi jelszó *

Add meg a jelenlegi jelszavad

Új jelszó *

Add meg az új jelszavad

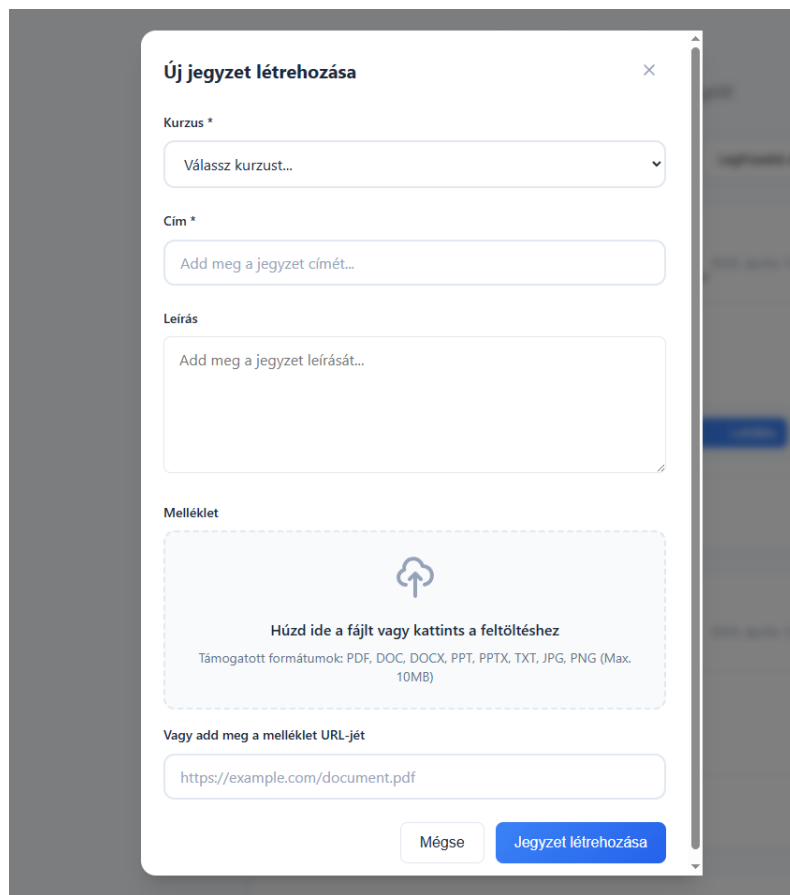
Jelszó megerősítése *

Add meg újra az új jelszavad

ábra 3.3-6 Beállítások

A beállítások oldalon a felhasználók megtekinthetik és módosíthatják fiókjuk adatait. Itt érhető el például az e-mail cím, a fiók létrehozásának dátuma, valamint lehetőség van a jelszó módosítására is.

3.3.7. Jegyzet létrehozása



ábra 3.3-7Jegyzet létrehozása

A jegyzet létrehozása felületen a felhasználók új tananyagokat tölthetnek fel a rendszerbe. A feltöltés során ki kell választani a megfelelő kurzust, meg kell adni a jegyzet címét és leírását, valamint csatolható különböző típusú fájlok, például PDF, DOCX vagy PPT formátumban.

A rendszer lehetőséget biztosít külső linkek megadására is, például Google Drive hivatkozások formájában. Ez növeli a rugalmasságot és megkönnyíti a különböző források integrálását.

4. Használt technológiák

4.1. React [1]

A React [1] egy széles körben használt JavaScript könyvtár, amelyet elsősorban felhasználói felületek fejlesztésére alkalmaznak. A React [1] komponens-alapú architektúrát követ, amely lehetővé teszi az alkalmazások moduláris felépítését. A

komponensek egymástól független, újrafelhasználható egységek, amelyek saját állapottal (state) és életciklussal rendelkeznek.

A komponensek újrafelhasználhatósága az egyik legnagyobb előnye a Reactnek. Egy jól megtervezett komponens több helyen is alkalmazható az alkalmazáson belül, ami csökkenti a kódismétlést és növeli a karbantarthatóságot. A komponensek közötti adatáramlás tipikusan „felülről lefelé” történik (props segítségével), ami kiszámíthatóbbá teszi az alkalmazás működését.

A React [1] a virtuális DOM [12] koncepcióját alkalmazza, amely optimalizálja a DOM-manipulációkat. A változások először egy memóriában lévő reprezentáción történnek, majd csak a szükséges módosítások kerülnek ténylegesen a böngésző DOM [12]-jába. Ez jelentősen javítja a teljesítményt nagyobb alkalmazások esetén.

4.1.1. Vite [13], Typescript [14]

A Vite [13] egy modern frontend build eszköz, amely gyors fejlesztői környezetet biztosít. A Vite [13] egyik legfontosabb jellemzője a villámgyors indítási idő és a hot module replacement (HMR) [15], amely lehetővé teszi a változtatások azonnali megjelenését a böngészőben újratöltés nélkül. A Vite [13] natív ES modulokat [16] használ fejlesztés közben, ami jelentősen csökkenti a build időt.

A TypeScript [14] egy statikusan típusos JavaScript [17] kiterjesztés, amely lehetővé teszi típusok definiálását. A TypeScript [14] használata növeli a kód megbízhatóságát és olvashatóságát, mivel már fordítási időben kiszűri a hibák egy részét. Nagyobb projektek esetén különösen hasznos, mert segíti a fejlesztők közötti együttműködést és a kód hosszú távú karbantartását.

A React [1] és TypeScript [14] kombinációja különösen erős, mivel a komponensek props és state struktúrái pontosan definiálhatók, így csökkenthető a futás idejű hibák száma.

4.1.2. Főbb frontend kiegészítő könyvtárak

Axios [18] egy Promise-alapú HTTP kliens, amelyet API hívások kezelésére használnak. Segítségével egyszerűen megvalósítható a backenddel való kommunikáció (GET, POST, PUT, DELETE kérések). Az Axios [18] támogatja az interceptorokat is, amelyek lehetővé teszik például autentikációs tokenek automatikus hozzáadását a kérésekhez.

A React Hook Form [19] egy hatékony megoldás űrlapok kezelésére React [1] alkalmazásokban. Minimalizálja az újrendereléseket, ezáltal javítja a teljesítményt. Egyszerű API-ja lehetővé teszi validációk kezelését és komplex űrlaplogika implementálását.

A React Router [20] a kliensoldali navigáció megvalósítására szolgál. Segítségével az alkalmazás különböző nézetei URL alapján jelennek meg, anélkül, hogy teljes oldalfrissítés történne. Ez az úgynevezett Single Page Application (SPA) [21] működés alapja.

4.1.3. Node.js [2]

A Node.js egy szerveroldali JavaScript [17] futtatókörnyezet, amely lehetővé teszi, hogy JavaScript nyelven backend alkalmazásokat is fejlesszünk. A Node.js [2] esemény vezérelt és nem blokkoló I/O modellt használ, ami különösen alkalmassá teszi nagy teljesítményű, skálázható hálózati alkalmazások fejlesztésére.

A Node.js [2] egyik legnagyobb előnye az egységes nyelvhasználat (JavaScript) [17] a frontend és backend oldalon, ami egyszerűsíti a fejlesztési folyamatot és csökkenti a kontextusváltást.

4.2. Express [3], TypeScript [14]

A Express egy minimalista és rugalmas webes keretrendszer Node.js-hez [2]. Segítségével gyorsan létrehozhatók REST API-k és webalkalmazások. Az Express middleware alapú architektúrát használ, ami lehetővé teszi a kérelmfeldolgozás lépéseinek láncolását. A middleware-ek olyan függvények, amelyek hozzáférnek a kéréshez (request), a válaszhoz (response), és a következő middleware-hez. Ez a modell rendkívül rugalmas és jól skálázható.

A TypeScript [5] használata backend oldalon is előnyös, mivel típusbiztonságot nyújt és segíti a hibák korai felismerését. Az API végpontok, adatmodellek és middleware-ek egyértelműen definiálhatók, ami növeli a kód minőségét.

4.2.1. Főbb backend kiegészítő könyvtárak

A fájlok feltöltésének kezelésére a Multer [22] middleware került alkalmazásra. Ennek segítségével a felhasználók különböző formátumú fájlokat (például PDF, DOCX, PPT) tudnak feltölteni a rendszerbe.

A felhasználói munkamenetek kezeléséhez cookie-alapú megoldás került alkalmazásra, amelyet a cookie-parser [23] middleware támogat. Ez biztosítja a felhasználók azonosítását a különböző kérések során.

A rendszer működésének naplózására a Winston [24] könyvtár szolgál, amely lehetővé teszi a szerveroldali események és hibák rögzítését. Ez segíti a hibakeresést és a rendszer stabil működésének biztosítását.

4.3. Miért ezeket a technológiákat választottam?

A fejlesztése során alkalmazott technológiák kiválasztása részben szakmai tapasztalat, részben pedig technológiai szempontok alapján történt. Fontos cél volt egy olyan technológiai stack kialakítása, amely egyszerre biztosít gyors fejlesztési lehetőséget, megbízható működést és hosszú távú bővíthetőséget.

A frontend fejlesztéshez a React [1] került kiválasztásra, amelyet korábbi projektek során már több éve használok. A technológiában szerzett tapasztalat lehetővé tette a hatékony és gyors fejlesztést. A React [1] komponens-alapú szemlélete különösen előnyösnek bizonyult, mivel jól strukturált, átlátható és újrafelhasználható felületi elemek kialakítását teszi lehetővé. Emellett a React [1] folyamatos fejlődése és aktív közössége biztosítja, hogy a technológia hosszú távon is releváns maradjon.

A build eszköz kiválasztásánál a Vite [13] mellett döntöttem, mivel a hagyományos React build megoldásokhoz képest lényegesen gyorsabb fejlesztési élményt nyújt. A Vite [13] gyors indítása és az azonnali visszajelzést biztosító működése jelentősen növeli a fejlesztés hatékonyságát, különösen nagyobb projektek esetén.

A TypeScript [14] használata tudatos döntés volt mind a frontend, mind a backend oldalon. A statikus típusellenőrzés növeli a kód biztonságát, csökkenti a futásidejű hibák számát, valamint átláthatóbbá teszi az adatstruktúrák kezelését. A modern webfejlesztésben a TypeScript [14] használata gyakorlatilag alapkövetelménynek tekinthető, különösen komplexebb rendszerek esetén.

A backend megvalósításához a Node.js [2] és az Express [3] keretrendszer került alkalmazásra. Ezekkel a technológiákkal szintén több éves tapasztalattal rendelkezem, ami lehetővé tette a gyors és hatékony fejlesztést. A Node.js [2] előnye az eseményvezérelt működés és a jó teljesítmény, míg az Express egyszerűsége és rugalmassága miatt ideális választás REST API-k kialakítására.

A kiegészítő könyvtárak kiválasztása funkcionalitás alapú volt. Minden alkalmazott eszköz egy konkrét problémára kínál megoldást, például HTTP

kommunikáció, űrlapkezelés, fájlfeltöltés vagy naplózás területén. Ez a moduláris megközelítés növeli a rendszer átláthatóságát és megkönnyíti a későbbi bővítést.

5. Rendszerarchitektúra

A Study Hub webalkalmazás egy háromrétegű (three-tier) architektúrát követ, amely a frontend (kliensoldali), backend (szerveroldali) és adatbázis rétegekből épül fel. Ez a felépítés biztosítja a rendszer modularitását, skálázhatóságát és karbantarthatóságát.

A rendszer működése során a felhasználó egy webböngészőn keresztül éri el az alkalmazást, ahol a frontend réteg kezeli a megjelenítést és az interakciókat. A frontend HTTP alapú kéréseken keresztül kommunikál a backend API-val, amely feldolgozza a kéréseket, végrehajtja az üzleti logikát, majd adatokat kér le vagy ír az adatbázisba.

5.1. Frontend architektúra

A frontend egy egyoldalas webalkalmazásként (Single Page Application) [21] került megvalósításra React [1] segítségével. A komponens-alapú architektúra lehetővé teszi a felhasználói felület moduláris felépítését, ahol az egyes funkciók különálló, újrafelhasználható komponensek formájában jelennek meg.

A felület felépítése során külön komponensek valósítják meg például:

- a navigációs sávot,
- a jegyzetek listázását,
- a kurzusok megjelenítését,
- valamint az űrlap alapú műveleteket (pl. bejelentkezés, jegyzet feltöltés).

A frontend főbb feladatai:

- felhasználói interakciók kezelése,
- adatok megjelenítése strukturált formában,
- űrlapok kezelése és validáció,
- API hívások indítása a backend felé.

Az alkalmazáson belüli navigáció kliensoldali routing segítségével történik, amely lehetővé teszi az oldalak közötti gyors váltást anélkül, hogy a teljes oldal újratöltődne.

A frontend és a backend közötti kommunikáció REST alapú API-n keresztül történik. Az adatcsere JSON [25] formátumban valósul meg, amely egyszerűen

feldolgozható és széles körben támogatott. A frontend architektúra kialakításánál fontos szempont volt a felhasználói élmény (UX), a gyors működés, valamint az átlátható komponensstruktúra.

5.2. Backend architektúra

A backend Node.js [2] környezetben, Express [3] keretrendszer segítségével került megvalósításra. A backend feladata a beérkező kérések feldolgozása, az üzleti logika megvalósítása, valamint az adatbázissal való kommunikáció.

A rendszer REST architektúrát követ, ahol az egyes funkciók külön végpontokon keresztül érhetők el. A backend szolgáltatja azokat az API végpontokat, amelyek lehetővé teszik többek között:

- felhasználói regisztráció és bejelentkezés,
- kurzusok és jegyzetek lekérdezése,
- jegyzetek feltöltése és kezelése,
- kommentek és értékelések kezelése.

A backend logikai felépítése rétegzett módon történt:

- **Route réteg:**
 - A HTTP végpontok definiálása, a kérések fogadása.
- **Controller réteg:**
 - Az üzleti logika megvalósítása, a kérések feldolgozása és a válaszok összeállítása.
- **Adatelérési réteg:**
 - Az adatbázis műveletek végrehajtása (lekérdezések, beszúrások, módosítások).

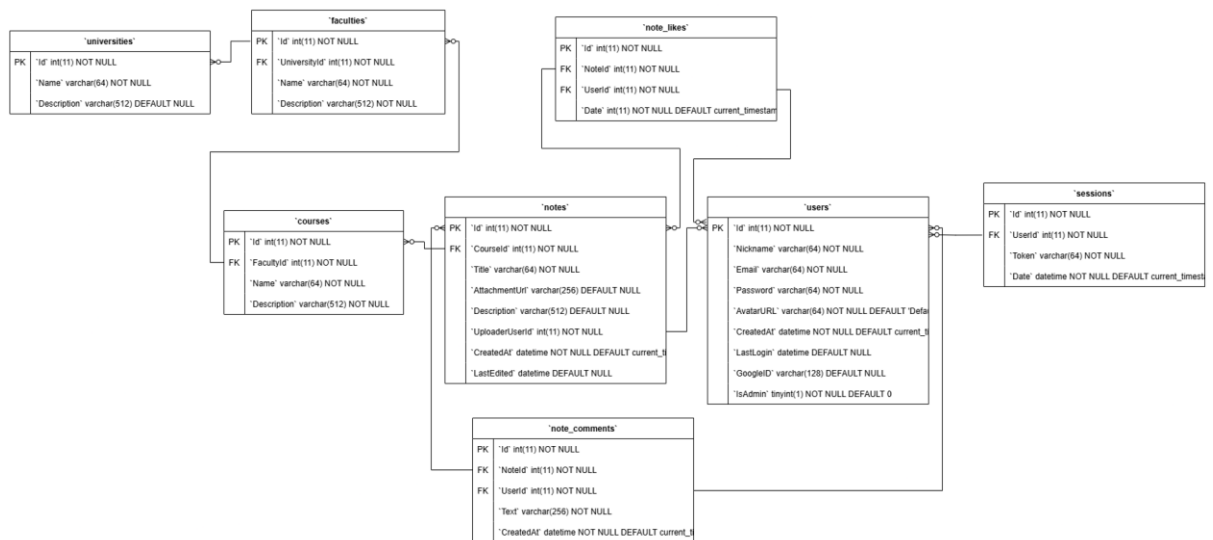
A middleware-ek használata lehetővé teszi a kérelmfeldolgozás különböző lépéseinek elkülönítését. Ilyen például:

- hitelesítés kezelése,
- fájlfeltöltés feldolgozása,
- naplózás.

A backend kezeli a felhasználói autentikációt is, amely biztosítja, hogy csak jogosult felhasználók férjenek hozzá bizonyos funkciókhoz (például jegyzet feltöltés vagy kommentelés).

5.3. Adatbázis architektúra

A rendszer adatainak tárolása relációs adatbázisban történik. Az adatbázis struktúrája egy Entity Relationship (ER) modell alapján került kialakításra, amely biztosítja az adatok közötti kapcsolatok egyértelmű kezelését.



ábra 5.3-1 Adatbázis ER diagram

Az adatmodell főbb entitásai:

- felhasználók (Users),
- egyetemek és karok (Universities, Faculties),
- kurzusok (Courses),
- jegyzetek (Notes),
- kommentek (Note_Comments),
- értékelések / kedvelések (Note_Likes),
- munkamenetek (Sessions).

Az entitások között különböző típusú kapcsolatok találhatók:

- 1:N kapcsolat, például egy kurzushoz több jegyzet tartozhat,
- N:1 kapcsolat, például egy jegyzet egy adott felhasználóhoz tartozik,
- N:M kapcsolat köztes táblával, például a felhasználók és jegyzetek közötti kedvelések esetén.

Az adatbázis tervezése során fontos szempont volt:

- az adatok redundanciájának minimalizálása,
- a normalizáció elveinek alkalmazása,
- az adatok konzisztenciájának biztosítása.

A megfelelő kulcsok (elsődleges és idegen kulcsok) használata biztosítja az entitások közötti kapcsolatok integritását. Az adatbázis struktúra lehetővé teszi a hatékony lekérdezéseket, valamint a rendszer későbbi bővítését új funkciókkal.

6. Kódrészletek

A rendszer működése szempontjából legfontosabb kódrészletek kerülnek bemutatásra, amelyek jól szemléltetik az alkalmazás főbb funkcióinak megvalósítását.

6.1. Felhasználó bejelentkezés

```
export const login = async (req: Request, res: Response) => {
  const { Email, Password } = req.body;
  const language = (req.headers.language as language) || "hu";

  const sessionToken = await authenticateUser(Email, Password);
  if (!sessionToken) return returnError(res, responses.Invalid_Username_Or_Password, language);

  await updateLastLogin(Email);

  res.cookie(COOKIE_NAMES.SESSION, sessionToken, {
    httpOnly: true,
    sameSite: "lax",
    secure: process.env.NODE_ENV === "production",
  });

  res.json({ success: true, message: responses.Login_Successful[language] });
};
```

ábra 6.1-1 Bejelentkezés endpoint

A fenti kódrészlet a felhasználói bejelentkezés backend oldali megvalósítását mutatja. A beérkező kérésből a rendszer kinyeri az email címet és a jelszót, majd az `authenticateUser` függvény segítségével ellenőrzi a hitelesítő adatokat.

Sikeres hitelesítés esetén a rendszer egy session tokenet generál, amely cookie formájában kerül eltárolásra a kliens oldalra. A cookie `httpOnly` beállítása biztosítja, hogy az ne legyen elérhető kliens oldali JavaScript számára, ezáltal növelve a biztonságot. A `secure` opció gondoskodik arról, hogy éles környezetben csak HTTPS kapcsolaton keresztül legyen továbbítva.

6.2. Jelszó módosítása

```
export const changePassword = async (req: Request, res: Response) => {
  const language = (req.headers.language as language) || "hu";
  const token = req.cookies?.sessiontoken;
  const { CurrentPassword, NewPassword, NewPasswordConfirm } = req.body;

  if (!CurrentPassword) return returnError(res, responses.Fill_The_Fields, language);
  if (!NewPassword) return returnError(res, responses.Fill_The_Fields, language);
  if (!NewPasswordConfirm) return returnError(res, responses.Fill_The_Fields, language);

  if (NewPassword !== NewPasswordConfirm) return returnError(res, responses.Passwords_Do_Not_Match, language);

  if (!token) return returnError(res, responses.You_Are_Not_Logged_In, language);

  const user = await getUserBySessionToken(token);

  if (!user) return returnError(res, responses.You_Are_Not_Logged_In, language);

  const isCorrectPassword = await checkPassword(user.Id, CurrentPassword);
  if (!isCorrectPassword) return returnError(res, responses.Invalid_Password, language);

  const changed = await updatePassword(user.Id, NewPassword);
  if (!changed) return returnError(res, responses.Unexpected_Error, language);

  return res.status(200).json({
    success: true,
    message: responses.Successfully_Changed_Password[language],
  });
};
```

ábra 6.2-1.Jelszó módosítás endpoint

Ez a kódrészlet a jelszó módosításának folyamatát mutatja be. A rendszer először ellenőrzi, hogy minden szükséges mező ki van-e töltve, majd összehasonlítja az új jelszót és annak megerősítését.

A felhasználó azonosítása a session token segítségével történik, amely a cookie-ból kerül kiolvasásra. A rendszer ezt követően ellenőrzi a jelenlegi jelszó helyességét, majd sikeres validáció esetén elvégzi az új jelszó mentését.

6.3. Jegyzet létrehozása

```
export const createNote = async (req: Request, res: Response) => {
  try {
    const language = (req.headers.language as language) || "hu";
    const id = parseInt(req.params.id);
    const { Title, AttachmentUrl, Description } = req.body;
    const token = req.cookies?.sessiontoken;

    if (!token) return returnError(res, responses.You_Need_To_Login_To_Use_This_Function, language);

    const noteId = await notesService.create(Title, AttachmentUrl, Description, token, id);

    return res.status(200).json({
      success: true,
      message: responses.Successfully_Created_Note[language],
      noteId: noteId,
    });
  } catch {
    return returnError(res, responses.Unexpected_Error, "hu");
  }
};
```

ábra 6.3-1 Jegyzet létrehozása endpoint

A kódrészlet a jegyzet létrehozásának backend oldali megvalósítását mutatja be. A rendszer a kérésből kiolvassa a jegyzet címét, leírását, valamint a csatolt fájl elérési útját.

A művelet végrehajtásának feltétele, hogy a felhasználó be legyen jelentkezve, amelyet a session token jelenléte biztosít. A tényleges létrehozási logika a notesService rétegben valósul meg, amely elválasztja az üzleti logikát a vezérlő rétegtől.

6.4. Hozzászólás írása jegyzethez

```
export const createComment = async (req: Request, res: Response) => {
  const language = (req.headers.language as language) || "hu";
  const noteId = parseInt(req.params.id, 10);
  const { content } = req.body;
  const token = req.cookies?.sessiontoken;
  if (!token) return returnError(res, responses.You_Need_To_Login_To_Use_This_Function, language);

  try {
    const commentId = await commentService.createComment(noteId, token, content);
    const comment = await commentService.getCommentById(commentId);
    return res.json({ success: true, comment });
  } catch {
    return returnError(res, responses.Unexpected_Error, "hu");
  }
};
```

ábra 6.4-1 Hozzászólás írása jegyzethez

A működés első lépése a jogosultság ellenőrzése: a rendszer csak bejelentkezett felhasználók számára engedélyezi a komment létrehozását. Ez a session token meglétének vizsgálatával történik, amely cookie formájában kerül továbbításra.

A tényleges üzleti logika a commentService rétegben kerül megvalósításra, amely biztosítja a felelősségi körök szétválasztását. A createComment metódus létrehozza az új hozzászólást az adatbázisban, majd annak azonosítójával lekérésre kerül a teljes komment objektum a getCommentById függvény segítségével.

Ez a megközelítés lehetővé teszi, hogy a kliensoldal azonnal megkapja a létrehozott komment minden szükséges adatát (például felhasználó, időbélyeg), így nem szükséges újra lekérni az összes hozzászólást. Ez javítja az alkalmazás teljesítményét és a felhasználói élményt. Hibakezeléshez try-catch blokkal történik, hogy megakadályozzuk a váratlan hibákat.

6.5. Kliensoldali navigáció (Routing)

```
return (
  <Routes>
    {!user ? (
      <>
        <Route path={PATHS.AUTH} element={<Navigate to={PATHS.LOGIN} />} />
        <Route path={PATHS.AUTH} element={<AuthLayout />} />
        <Route path={PATHS.LOGIN} element={<Login />} />
        <Route path={PATHS.REGISTER} element={<Register />} />
      </Route>
    ) : (
      <>
        <Route path="/auth/*" element={<Navigate to={PATHS.HOME} />} />
      </>
    )}
    <Route element={<Layout isAuthenticated={!user} />}>
      <Route path={PATHS.HOME} element={<Home />} />
      <Route path={PATHS.COURSES} element={<Courses />} />
      <Route path={PATHS.COURSES + ":id"} element={<Course />} />
      <Route path={PATHS.NOTES + ":id"} element={<NotePage />} />
      <Route path={PATHS.SETTINGS} element={<Settings user={user as User} />} />
    </Route>
    <Route path="*" element={<Navigate to={PATHS.HOME} />} />
  </Routes>
);
```

ábra 6.5-1 React routing

A kódrészlet a frontend oldali navigáció megvalósítását mutatja be React Router [20] segítségével. Az alkalmazás különböző útvonalakon keresztül jeleníti meg az egyes oldalakat, például a főoldalt, kurzusokat vagy a jegyzetek részleteit. A routing logika figyelembe veszi a felhasználó bejelentkezési állapotát is. Amennyiben a felhasználó nincs bejelentkezve, csak az autentikációs oldalak érhetők el. Bejelentkezett állapotban a felhasználó hozzáfér a teljes alkalmazáshoz.

6.6. API kommunikáció

```
class CoursesService {
  async getUniversities(): Promise<University[]> {
    try {
      const response = await axios.get(ENDPOINTS.UNIVERSITIES);
      return response.data;
    } catch (error) {
      console.error("Error fetching universities:", error);
      throw error;
    }
  }

  async getUniversityFaculties(universityId: number): Promise<Faculty[]> {
    try {
      const response = await axios.get(ENDPOINTS.UNIVERSITY_FACULTIES(universityId));
      return response.data;
    } catch (error) {
      console.error(`Error fetching faculties for university ${universityId}:`, error);
      throw error;
    }
  }

  async getFacultyCourses(facultyId: number): Promise<Course[]> {
    try {
      const response = await axios.get(ENDPOINTS.FACULTY_COURSES(facultyId));
      return response.data;
    } catch (error) {
      console.error(`Error fetching courses for faculty ${facultyId}:`, error);
      throw error;
    }
  }
}
```

ábra 6.6-1 Api kommunikáció

A kódrészlet a frontend oldali API kommunikáció egy példáját mutatja be. A CoursesService osztály felelős a backenddel való adatlekérésért, például egyetemek, karok és kurzusok esetében.

Az API hívások Axios [18] segítségével történnek, amely egyszerű és átlátható módot biztosít HTTP kérések kezelésére. A service réteg alkalmazása lehetővé teszi a hálózati logika elkülönítését a felhasználói felülettől, ami növeli a kód átláthatóságát és újrafelhasználhatóságát.

7. Telepítési útmutató

7.1. Környezet

A projekt futtatásához szükséges egy megfelelő fejlesztői környezet. Szükségünk van egy telepített Node.js [2] környezetre. Minimum a 18-as verzió ajánlott, azonban érdemes frissebb, stabilabb LTS verziót használni (Pl.: 24.15.0)

Csomagkezeléshez az npm javasolt, mivel a projekt is Windows környezetben készült.

Adatbázis működéséhez szükségünk van egy MySQL [4] szerverre. A fejlesztés során phpMyAdmin [26] került felhasználásra, amely jelentősen könnyíti az adatbázis kezelését.

A projekt két fő részből áll:

Frontend

Backend

7.2. Backend beállítása

A backend működéséhez először az adatbázist kell előkészíteni.

Indítsuk el a XAMPP Control Panelt, majd az Apache és MySQL [4] szolgáltatásokat.

Nyissuk meg a phpMyAdmin [26] felületet (általában: <http://localhost/phpmyadmin>).

Hozzunk létre egy új adatbázist studyhub néven.

Importáljuk a projektben található .sql fájlt az adatbázisba.

Sikeres importálás után következik a backend konfigurálása.

Navigáljunk az api mappába, majd hozzunk létre egy .env fájlt az alábbi tartalommal:

```
MYSQL_HOST=localhost  
MYSQL_USER=root  
MYSQL_PASS=  
MYSQL_DB=studyhub  
MYSQL_PORT=3306  
APP_URL=http://localhost:5173  
PORT=2004
```

Ezután telepítsük a szükséges csomagokat:

```
npm install
```

A backend indítása fejlesztői módban:

```
npm run dev
```

Amennyiben buildelt verziót szeretnénk készíteni:

```
npm run build
```

A fejlesztés során a dev script használata javasolt, mivel ez automatikus újratöltést (hot reload) biztosít.

7.3. Frontend beállítása

A frontend beállítása hasonlóan történik, és viszonylag egyszerű.

Navigáljunk az app mappába, majd telepítsük a csomagokat:

```
npm install
```

Alternatív megoldásként használható az:

```
npm ci
```

Ez a package-lock.json fájlban rögzített verziókat telepíti, ami biztosítja a konzisztens környezetet, azonban bizonyos esetekben elavult csomagokat is tartalmazhat.

Ezután hozzuk létre egy .env fájlt az alábbi tartalommal:

```
VITE_API_URL=http://localhost:2004/api/
```

```
VITE_APP_URL=http://localhost:5173/
```

A frontend indítása fejlesztői módban:

```
npm run dev
```

Build készítése

```
npm run build
```

Fejlesztés során a dev script használata ajánlott, mivel a backendhez hasonlóan hot reload funkcióval rendelkezik, így a változtatások azonnal megjelennek a böngészőben.

Rendszer elérése

A rendszer sikeres indítás után az alábbi címen lesz elérhető:

```
http://localhost:5173/
```

A backend a következő címen:

```
http://localhost:2004/
```

Fontos, hogy a projekthez a backend, és a frontend is egyszerűen fusson. Ha probléma adódik az adatbázis, és a backend kapcsolattal, akkor ellenőrizzük a .env fájlban megadott adatokat.

8. Továbbfejlesztési lehetőségek

Tervezés során már odafigyeltem, hogy olyan funkciókat találjak ki, olyan funkciókat hozzak létre, melyek könnyen bővíthetők. Funkcionalitás szempontból nem

érzem, hogy nehéz helyzetben lennék, mivel rengeteg oktatással, és szociális médiával foglalkozó termék/szolgáltatás létezik már. Tervezésnél is már úgy alakítottam ki a struktúrát, hogy ne okozzon problémát új funkciók implementálása, esetleg kisebb refaktorálások ne okozzon gondot.

Az egyik legjelentősebb továbbfejlesztési irány a mesterséges intelligencia integrálása lehet. Az MI alkalmazása az oktatás területén egyre nagyobb szerepet kap, és a Study Hub esetében is számos hasznos funkció alapját képezheti. Például lehetőség lenne arra, hogy a rendszer automatikus összefoglalókat készítsen a feltöltött jegyzetekből. Egy dokumentum tartalmának elemzésével a rendszer kiemelhetné a legfontosabb információkat, és egy rövid, könnyen áttekinthető kivonatot jeleníthetne meg a felhasználók számára.

További lehetőség az úgynevezett „tanulókártya” (flashcard) alapú tanulási rendszer bevezetése. Ebben az esetben a feltöltött anyagokból automatikusan generált kérdés-válasz párok segítenék a tanulást, különösen olyan területeken, mint például idegen nyelvek vagy definíciók elsajátítása. Ez jelentősen növelné a platform interaktivitását, és aktív tanulási folyamatot támogatna.

Mivel szinte mindenkinek van mobiltelefonja, ezért úgy gondolom, hogy mindenképpen szükség lenne egy mobil alkalmazásra. Sokkal könnyebb egy alkalmazást megnyitni, mint egy weboldalt, ez mellett sokkal több felhasználó látogatja az oldalt telefonról, mint számítógépről. Számtalan új lehetőséget is adna egy mobil alkalmazás, például push értesítések bevezetése, amelyekkel értesíthetnénk a felhasználót új jegyzetéről, kommentekről, vagy egyéb más aktivitásokról.

9. Mesterséges intelligencia használata a fejlesztés során

9.1. Alkalmazott eszközök

A fejlesztés során elsősorban a ChatGPT, [27] valamint a GitHub Copilot [28] került használatra. A ChatGPT-t [27] főként magasabb szintű feladatokhoz használtam, mint például architektúra javaslatok, problémák elemzése, illetve dokumentáció készítése. A GitHub Copilot [28] ezzel szemben inkább a kódírás során nyújtott segítséget, például ismétlődő kódrészletek gyors generálásában vagy egyszerűbb logikai struktúrák megvalósításában.

Az egyes eszközök használata tehát jól elkülönült: a ChatGPT [27] inkább gondolkodási és tervezési támogatást adott, míg a Copilot [28] a fejlesztés során, közvetlenül a kódírás közben segített.

9.2. Felhasználási területek

Az MI eszközök több különböző feladattípusban kerültek alkalmazásra.

Kódgenerálás során főként egyszerűbb, jól ismert minták alapján működő részeknél használtam, például API végpontok alapstruktúrájának létrehozásánál, vagy frontend komponensek kezdeti felépítésénél. Ezeket azonban minden esetben módosítottam és a projekt specifikus igényeihez igazítottam.

A kódreview és refaktorálás során az MI segítségével alternatív megoldásokat is megvizsgáltam, különösen olyan esetekben, amikor többféle implementáció is lehetséges volt. Ez segített a tisztább és átláthatóbb kódszerkezet kialakításában.

Hibakeresés és debuggolás során az MI különösen hasznosnak bizonyult. Több esetben segített gyorsan azonosítani a problémák lehetséges okait, például hibás API hívások, rosszul kezelt aszinkron műveletek vagy típushibák esetén.

Az architektúra tervezés során az MI inkább ötletadó szerepet töltött be. Segített különböző megközelítések összehasonlításában, például a rétegezett backend struktúra vagy a service alapú felépítés kialakításában.

A dokumentáció készítése során is alkalmaztam MI eszközöket, különösen a szöveg megfogalmazásának finomítására és strukturálására. Azonban a tartalmi részek minden esetben saját megértésem alapján alakultak.

9.3. Validáció és ellenőrzés

Az MI által generált válaszokat minden esetben hipotézisként kezeltem, és nem fogadtam el automatikusan. A kapott megoldásokat saját tudás alapján, valamint gyakorlati teszteléssel ellenőriztem.

A validáció fő módszerei a következők voltak:

- a kód futtatása és tesztelése,
- hibakezelési esetek kipróbálása,
- dokumentációk (pl. hivatalos library dokumentációk) ellenőrzése,
- a kód logikai átgondolása.

Több esetben előfordult, hogy az MI által javasolt megoldás nem volt megfelelő. Például egyes esetekben a generált kód nem kezelte megfelelően az aszinkron műveleteket, ami hibás működéshez vezetett. Egy másik esetben a javasolt autentikációs megoldás nem volt biztonságos, mivel nem kezelte megfelelően a session állapotot. Ezeket a problémákat manuálisan javítottam, és a megoldást a projekt követelményeihez igazítottam.

Ez a tapasztalat megerősítette, hogy az MI eszközök használata során elengedhetetlen a kritikus gondolkodás és az önálló ellenőrzés.

9.4. MI használatának korlátai

Fontos kiemelni, hogy a fejlesztés bizonyos részeiben szándékosan nem használtam MI eszközöket. Ilyen volt például a rendszer alaplogikájának kialakítása, az adatbázis struktúra megtervezése, valamint a főbb architektúrai döntések meghozatala.

Ezeket a részeket tudatosan saját megértés alapján alakítottam ki, mivel ezek jelentik a rendszer alapját, és fontos volt, hogy teljes mértékben átlássam azok működését.

A szakdolgozat szövegének tartalmi részei szintén saját gondolatokon alapulnak, az MI eszközök itt inkább a megfogalmazás finomításában segítettek.

9.5. Tapasztalatok és tanulságok

Az MI eszközök jelentősen gyorsították a fejlesztési folyamatot, különösen az ismétlődő feladatok és a jól ismert minták implementálása során. Emellett segítettek új megközelítések megismerésében és a problémák gyorsabb megértésében.

Ugyanakkor bizonyos esetekben az MI félrevezető is lehet, különösen akkor, ha a generált megoldás látszólag helyes, de valójában nem illeszkedik a konkrét problémához. Ezért a jövőben is fontosnak tartom, hogy az MI-t támogató eszközként, és ne helyettesítőként használjam.

A fejlesztési folyamat során szerzett tapasztalatok alapján a jövőben még tudatosabban alkalmaznám az MI eszközöket, különösen a komplexebb problémák elemzése és a kódminőség javítása érdekében.

Irodalomjegyzék

- [1] „React,” [Online]. Available: <https://react.dev>.
- [2] „Node.js,” [Online]. Available: <https://nodejs.org/>.
- [3] „Express.js,” [Online]. Available: <https://expressjs.com/>.
- [4] „MySQL,” [Online]. Available: <https://www.mysql.com/>.
- [5] „Visual Studio Code,” [Online]. Available: <https://code.visualstudio.com/>.
- [6] „Notion,” [Online]. Available: <https://www.notion.com/>.
- [7] „Google Classroom,” [Online]. Available: <https://edu.google.com/workspace-for-education/products/classroom/>.
- [8] „Studocu,” [Online]. Available: <https://www.studocu.com/>.
- [9] „Studydrive,” [Online]. Available: <https://www.studydrive.net/>.
- [10] „Figma,” [Online]. Available: <https://www.figma.com/>.
- [11] „Google,” [Online]. Available: <https://www.google.com/>.
- [12] „Document Object Model (DOM),” [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/API/Document_Object_Model.
- [13] „Vite,” [Online]. Available: <https://vite.dev/>.
- [14] „Typescript,” [Online]. Available: <https://www.typescriptlang.org/>.
- [15] „Hot Module Replacement,” [Online]. Available: <https://webpack.js.org/concepts/hot-module-replacement/>.
- [16] „ES module,” [Online]. Available: https://www.w3schools.com/nodejs/nodejs_modules_esm.asp.
- [17] „Javascript,” [Online]. Available: <https://developer.mozilla.org/en-US/docs/Web/JavaScript>.
- [18] „Axios,” [Online]. Available: <https://github.com/axios/axios>.
- [19] „React Hook Form,” [Online]. Available: <https://react-hook-form.com/>.
- [20] „React Router,” [Online]. Available: <https://reactrouter.com/>.
- [21] „SPA (Single-page application),” [Online]. Available: <https://developer.mozilla.org/en-US/docs/Glossary/SPA>.
- [22] „Multer,” [Online]. Available: <https://www.npmjs.com/package/multer>.
- [23] „Cookie Parser,” [Online]. Available: <https://www.npmjs.com/package/cookie-parser>.
- [24] „Winston,” [Online]. Available: <https://www.npmjs.com/package/winston>.
- [25] „JSON,” [Online]. Available: <https://www.json.org/json-en.html>.
- [26] „phpMyAdmin,” [Online]. Available: <https://www.phpmyadmin.net/>.
- [27] „ChatGPT,” [Online]. Available: <https://chatgpt.com/>.

[28] „Github Copilot,” [Online]. Available: <https://github.com/copilot>.

Nyilatkozat

Alulírott Kökény Szabolcs programtervező informatikus BSc szakos hallgató, kijelentem, hogy a dolgozatomat a Szegedi Tudományegyetem, Informatikai Intézet Szoftverfejlesztési Tanszékén készítettem, programtervező informatikus BSc diploma megszerzése érdekében.

Kijelentem, hogy a dolgozatot más szakon korábban nem védtem meg, saját munkám eredménye, és csak a hivatkozott forrásokat (szakirodalom, eszközök, stb.) használtam fel. Tudomásul veszem, hogy szakdolgozatomat / diplomamunkámat a Szegedi Tudományegyetem Diplomamunka Repozitóriumban tárolja.

Szeged, 2026.04.30.

Kökény Szabolcs

Köszönetnyilvánítás

Elsősorban köszönettel tartozom témavezetőmnek, Dr. Bilicki Vilmos egyetemi adjunktusnak a szakmai útmutatásért, hasznos tanácsokért és a dolgozat elkészítése során nyújtott támogatásáért. Irányítása és visszajelzései nagyban hozzájárultak ahhoz, hogy a munkát megfelelő irányba tudjam fejleszteni.

Köszönet illeti családomat és barátaimat is, akik a tanulmányaim során végig támogattak, és motiváltak a nehezebb időszakokban is.